

# System Architecture

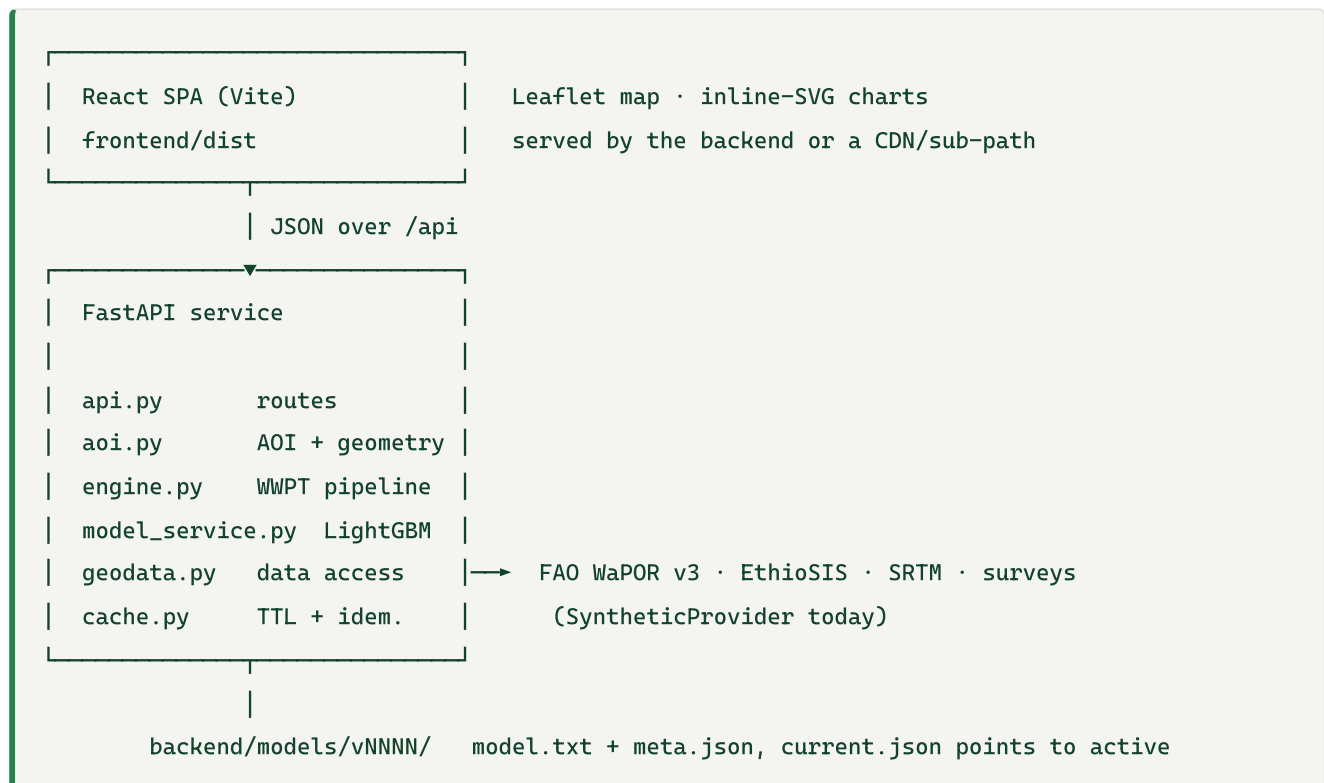
## Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II  
Generated 2026-07-29

## Architecture

### Overview

Two deployable pieces and one artifact store:



The frontend never talks to WaPOR or the model directly; every value it renders comes from one of the nine `/api` routes. That boundary is what lets the data provider be swapped without touching the UI.

### Request flow: one analysis run

1. **POST `/api/analysis`** with the AOI spec, production system, year and season. Pydantic validates types; `api._validate_season` rejects combinations the system cannot produce (for example Meher under irrigation).
2. **Idempotency and caching** (`cache.py`). If an `Idempotency-Key` header matches a completed run, that exact response is replayed — a retried POST never starts a second computation. Otherwise the

canonical request JSON is hashed and checked against a 30-minute TTL cache; a hit returns the stored result with `cached: true`.

3. **AOI resolution** (`aoi.resolve`). Admin units come from the boundary layer; drawn polygons are used directly; uploads are looked up by id in the upload registry. Anything unresolvable raises `AOIError`, surfaced as HTTP 422 with a message written for the person using the dashboard, not the developer.
4. **Grid and mask** (`engine.run_analysis`). A 170×170 lat/lon grid is built over the AOI bounds, intersected with the AOI geometry (vectorized even-odd point-in-polygon) and with the wheat-area mask.
5. **Feature assembly** (`geodata.PROVIDER.assemble`). Ten features are produced for every unmasked cell as numpy arrays — one vectorized call, not per-cell.
6. **Inference** (`model_service.MODEL.predict`). One LightGBM call over the whole (n, 10) matrix.
7. **Products**. The values are colourized and encoded as a PNG data-URI (`pnglib.py`, stdlib `zlib` only — no image dependency), plus zonal statistics, the class distribution, a five-season trend computed on a coarser 42×42 grid, and current feature importance. The run is registered under a `run_id` so CSV export can reproduce it.

Pixel inspection (`/predict`) and explanation (`/explain`) run the same feature assembly for a single coordinate, so a pixel's popup value and its explanation are always consistent with the raster around it.

## Data-access layer

`geodata.SyntheticProvider` is the seam between the analytics and the outside world. It exposes one method the engine depends on:

```
assemble(lat, lon, system, year, season) -> dict[str, np.ndarray]
```

The synthetic implementation builds each field from sums of harmonics, so values are deterministic (the same coordinates always give the same answer), spatially coherent (neighbouring cells correlate, as real remote-sensing products do), and physically ordered (elevation descends toward the Afar lowlands, Meher is wetter than Belg, drought years score below 1). NPP additionally carries ~9% retrieval noise uncorrelated with the other layers, which is what stops the model from treating NPP as a perfect proxy and collapsing every explanation onto one feature.

**To go live with WaPOR**, implement the same signature against the FAO WaPOR v3 retrieval for NPP and AET, plus the ancillary layers, and set `PROVIDER`. Nothing in `engine.py`, `model_service.py`, `api.py` or the frontend changes.

`true_wwp()` exists only to label synthetic training samples; a real deployment trains on measured field observations instead and can delete it.

## Model service

**Artifacts.** `backend/models/vNNNN/` holds `model.txt` (the LightGBM booster) and `meta.json` (version, timestamp, holdout metrics, training provenance). `current.json` names the active version. On startup the service loads the active artifact, or trains v0001 if none exists.

**Inference.** `predict()` clamps to the physically plausible 0.15–2.2 kg/m<sup>3</sup> band.

**Explanations.** `explain()` uses LightGBM's native `pred_contrib`, which returns exact tree-path SHAP values plus the base value — no sampling and no extra dependency. Contributions sum to the prediction, so the waterfall the dashboard draws is arithmetically honest; the UI shows the seven largest by magnitude and the table view lists their values and units.

**Importance** is reported as *split* count, not gain. With strongly correlated drivers (NPP, AET and rainfall all encode water supply) gain assigns ~100% to whichever feature wins the first split and leaves everything else near zero, which reads as "only NPP matters" and is misleading. Split count reflects how often the model actually consults each feature and produces the graded profile users can act on.

**Retraining** ( `POST /api/model/retrain` ) is the workflow the ToR asks for: EIAR uploads a CSV of new field observations, the service blends them with the base development set, trains a candidate, and scores **both** the candidate and the currently active model on the *same* holdout split. The candidate is promoted only if its RMSE is within 2% of or better than the active model's; otherwise the active model stays serving and the response explains why. Promotion writes a new version directory and flips `current.json`, so the swap is atomic and the previous version remains on disk to roll back to. `force=true` overrides the guard.

## Visualization palette

Every chart colour was chosen by job and validated, not by eye. The validator checks lightness ordering, chroma, colour-vision-deficiency separation and contrast against the surface.

| JOB                                 | CHART                               | TOKENS                                                                                                   |
|-------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| Sequential / ordinal<br>(magnitude) | map raster, legend,<br>distribution | <code>#93bd82</code> <code>#6ba763</code> <code>#458b4b</code> <code>#297038</code> <code>#0f4d26</code> |
| Single series (identity)            | trend line, feature<br>importance   | <code>#297038</code>                                                                                     |
| Diverging (polarity)                | prediction explanation              | <code>#d97a1e</code> raises · <code>#1f6f9c</code> lowers, neutral grey zero<br>line                     |

Two deliberate departures from the original HTML prototype:

- **The WWP ramp is single-hue green, not brown→yellow→green.** The prototype's ramp put a pale yellow ( `#f7dd72` ) at the middle of the scale, which measures 1.32:1 against a white card — effectively invisible in the distribution chart — and its lightness was non-monotone, so the ramp did not read as ordered under colour-vision deficiency or in greyscale print. A single green hue, light to dark, keeps the EIAR identity, encodes magnitude correctly, and clears the 2:1 light-end contrast floor. The backend `engine.RAMP` and the frontend `charts.RAMP_HEX` hold the same five values, so the map, the legend and the chart cannot drift apart.
- **The explanation chart uses orange/blue, not green/red.** Green and red collapse to  $\Delta E$  3.1 under protanopia — far below the  $\Delta E$  8 target — so a red-green pair would make "raises" and "lowers" indistinguishable for a substantial share of users. Orange against blue measures  $\Delta E$  21.4. Direction is additionally carried by which side of the zero line the bar sits on, a signed numeric label, and a legend, so colour is never the only channel.

The ramp spans a fixed 0.4–1.6 kg/m<sup>3</sup> rather than stretching to each result's own range, so two worded as can be compared directly across runs. The cost is lower contrast within a single scene; for a monitoring

tool comparability is worth more.

Every chart also ships a **Show table** toggle rendering the same numbers as text, and a hover tooltip. No value is reachable only by hovering, which keeps the figures usable by keyboard and screen-reader users and satisfies the ToR's accessibility requirement.

## Frontend

React with Leaflet driven imperatively. `MapView.jsx` owns every Leaflet call; React state flows in as props and Leaflet layers are reconciled in effects. Two details worth keeping:

- **The drawing preview updates in place.** Recreating the polyline and vertex markers on each new vertex changes the DOM node under the pointer, and Chromium will not emit `dblclick` when the target changes between the two clicks — so "double-click to finish" silently did nothing. The preview now mutates one persistent polyline via `setLatLngs` and appends only new markers.
- **Finishing a polygon has a button, not only a double-click.** Double-click is unavailable to keyboard users and awkward on touch, so `Finish polygon` is the primary control and the double-click is a shortcut.
- **The map watches its own container for resizes.** Leaflet only listens for *window* resizes. The results panel mounting or unmounting resizes `.mapwrap` through the flex layout without any window event, which would leave the map rendering at its stale width — blank tile gutters and a misplaced raster overlay — until the next pan. A `ResizeObserver` on the map element calls `invalidateSize()` instead, which also covers the responsive breakpoint.

The results panel is **mounted only once a run has produced results**, so it never occupies the layout while empty and the map gets the full width until there is something to show. Closing it returns that width and leaves a `Show results` control on the map, so dismissing the panel cannot strand a completed run.

Charts are hand-written inline SVG rather than a charting library: four small fixed-purpose figures cost less to draw directly than the ~150 kB a general charting bundle would add, and it keeps full control of the accessibility attributes and the mark specs.

## Deployment notes

Single process serves both tiers: `uvicorn app.main:app` mounts `frontend/dist` at `/` when it exists, with the API at `/api`. `vite.config.js` sets `base: './'`, so the built bundle works unchanged from any sub-path of the EIAR site ( `/research-tools/wwp/`, say) behind a reverse proxy.

For production hardening beyond this build, see the [ToR Coverage document](#) § What deployment still needs — chiefly PostGIS for the upload registry and run store (both in-memory today), Redis for the cache if more than one worker runs, and the live WaPOR provider.